TÀI LIỆU

KIẾN THỨC NỀN TẢNG VÀ NÂNG CAO

HỌC SINH GIỎI

TIN HỌC

THPT

Căn cứ 24 đề - 96 bài HSG THPT 2025-2026

Đã bổ sung 6 đề mới và các kỹ thuật phát sinh

Lê Thanh Nam

```
for (int i = 1; i <= n; ++i)
  sum += a[i];

if (x < mid)
  r = mid + 1;
else
  l = mid + 1;
```

$$dp[i][j] = dp[i-1][j] + w_i$$

$$S(n) = \sum_{i=1}^{n} a_i$$



TƯ DUY

CẤU TRÚC DỮ LIỆU

THUẬT TOÁN

LẬP TRÌNH C++

BÀI TOÁN THỰC CHIẾN

LỜI MỞ ĐẦU

Tin học thi đấu không phải là việc ghi nhớ thật nhiều đoạn mã. Cốt lõi của môn học là năng lực mô hình hóa: đọc một tình huống, nhận ra cấu trúc toán học ẩn phía sau, chọn biểu diễn phù hợp, rồi biến ý tưởng ấy thành một thuật toán có thể chứng minh và cài đặt chính xác. Vì vậy, một giáo trình dành cho học sinh giỏi không nên bắt đầu từ "mẹo giải nhanh", mà phải xây từ cách suy nghĩ đúng.

Bản cập nhật này được xây dựng trên nền tảng đề thi học sinh giỏi Tin học THPT năm học 2025-2026, gồm 96 bài toán. Trong đó, 18 đề đầu tạo nên khung kiến thức ban đầu; 06 đề bổ sung từ Lào Cai, Lâm Đồng, Phú Thọ, Quảng Ngãi, Tuyên Quang và Vĩnh Long giúp kiểm tra lại độ phủ của giáo trình và làm xuất hiện thêm một số mảng kiến thức cần trình bày riêng.

MỤC TIÊU BIÊN SOẠN

Học sinh chưa biết hoặc còn hổng kiến thức vẫn có thể bắt đầu từ Phần 0; học sinh đã có nền tảng có thể dùng tài liệu như một bản đồ kỹ thuật. Mỗi kỹ thuật được trình bày theo cùng một tuyến sư phạm: ví dụ cụ thể → cách làm trực tiếp → điểm bế tắc → quan sát then chốt → thuật toán tối ưu → chứng minh → code C++ → lỗi thường gặp → bài tập vận dụng.

Cách học để tài liệu thực sự có tác dụng

Bước 1. Đọc ví dụ nhỏ và tự tính đáp án bằng tay. Nếu chưa tính được bằng tay, chưa nên đọc code.

Bước 2. Viết cách làm trực tiếp nhất, dù chậm. Bản chậm giúp hiểu đúng bài và là "trọng tài" để kiểm thử bản tối ưu.

Bước 3. Thay giới hạn lớn nhất vào độ phức tạp. Phải chỉ ra chính xác thao tác nào đang bị lặp lại quá nhiều.

Bước 4. Tìm tính chất cho phép loại bỏ phần việc thừa: tính đơn điệu, tổng tiền tố, bất biến, cấu trúc cây, trạng thái DP, quan hệ thứ tự...

Bước 5. Chứng minh trước khi tối ưu code. Một lời giải nhanh nhưng không có lập luận đúng đắn vẫn là một lời giải chưa hoàn chỉnh.

Bước 6. Gõ lại chương trình, chạy tay trên ví dụ, rồi stress test bằng bản vét cạn khi có thể.

Tài liệu không có biến mọi bài toán thành một "công thức thuộc lòng". Ngược lại, mục tiêu là giúp người học nhìn thấy mối liên hệ giữa các bài: tổng tiền tố và sai phân là hai phép biến đổi ngược nhau; Fenwick và segment tree là bước nâng cấp khi dữ liệu thay đổi; chặt nhị phân chỉ hợp lệ khi tính khả thi đơn điệu; DSU và Kruskal cùng dựa trên quá trình hợp nhất thành phần; suffix array, rolling hash và LCP đều trả lời câu hỏi so sánh xâu con nhưng bằng những mô hình khác nhau.

NGUYÊN TẮC CỦA CUỐN SÁCH

Không chỉ biết "dùng thuật toán gì", học sinh phải trả lời được bốn câu:

- Vì sao nghĩ tới nó?
- Trạng thái/bất biến là gì?
- Vì sao nó đúng?
- Khi nào không được dùng?

Lê Thành Nam

MỤC LỤC

PHẦN 0 - C++ TỪ SỐ 0

0.1. Một chương trình C++ tối thiểu gồm những gì?
0.2. Biến, kiểu dữ liệu và phép gán
0.3. Nhập xuất với cin và cout
0.4. Điều kiện if / else
0.5. Vòng lặp for và while
0.6. Hàm - chia bài toán thành các khối nhỏ
0.7. Mảng, vector và string
0.8. Cách đọc một đề thi lập trình
0.9. Chạy tay, debug và kiểm thử

PHẦN I - NỀN TẢNG THPT

Chương 1. Tư duy giải thuật, độ phức tạp và C++ thi đấu
Chương 2. Mảng, tần suất, tổng tiền tố và mảng sai phân
Chương 3. Sắp xếp, tìm kiếm nhị phân và hai con trỏ
Chương 4. Cửa sổ trượt, ngăn xếp và deque đơn điệu
Chương 5. Xâu ký tự, bảng tần suất, phân tích cú pháp và tập vị tự
Chương 6. Số học cơ bản, sàng, phân tích thừa số và tổ hợp
Chương 7. Đồ thị cơ bản, BFS/DFS và cây
Chương 8. Quy hoạch động nền tảng và thiết kế trạng thái

PHẦN II - KỸ THUẬT NÂNG CAO THPT

Chương 9. Nén tọa độ, Fenwick tree và segment tree
Chương 10. DSU, Kruskal, kết nối offline và gộp nhỏ vào lớn
Chương 11. Đường đi ngắn nhất: Dijkstra và mở rộng trạng thái
Chương 12. Quy hoạch động nâng cao: interval, digit, cân bằng và tối ưu bộ nhớ
Chương 13. Chặt nhị phân trên đáp số và hàm kiểm tra đơn điệu
Chương 14. Tham lam, hàng đợi ưu tiên và sweep line
Chương 15. Xâu nâng cao: rolling hash, fingerprint ngẫu nhiên và bitset
Chương 16. Số học nâng cao: điều hòa, phần không chính phương, Möbius và bao hàm-loại trừ
Chương 17. Tối ưu toán học: trung vị, biến đổi và bài toán gán
Chương 18. Các khuôn chuyên sâu: constructive, cấu trúc động và chia để trị không gian
Chương 19. DP bitmask và bài toán người du lịch (TSP)
Chương 20. Persistent segment tree và truy vấn k-th / trung vị trên đoạn
Chương 21. Suffix array, LCP và xâu con lặp dài nhất
Chương 22. Cây nâng cao: LCA, binary lifting và truy vấn đường đi
Chương 23. Nén trạng thái GCD, số lớn chính xác và các bài đếm trên đoạn
Chương 24. Hộp công cụ mới: prime counting, comparator ghép chuỗi, prefix 2D và bottleneck graph

Lê Thanh Nam

TỔNG QUAN

Bảng dưới đây giúp dùng tài liệu như một "mục lục ngược": khi đang đọc một lời giải đề tỉnh, có thể tra nhanh chương kiến thức tương ứng. Một bài có thể liên quan nhiều chương; cột gợi ý chỉ nêu nhóm kỹ thuật chính.

Đơn vị	Các bài toán	Chương liên quan
Bắc Ninh	Độ lệch • Xóa số • Mật khẩu • Dãy con tăng	4, 5, 9, 16
TP. Cần Thơ	Khu vườn • Dãy đèn • Dãy số • Vòng tròn • Mạng máy tính	1, 2, 3, 7, 10, 16, 18
TP. Hải Phòng	Xuân đảo ngược • Số nguyên đặc biệt • Tìm cặp số • Xóa xâu • Tìm kiếm	3, 4, 5, 6
Hà Tĩnh - Khối 11	Số đẹp • Dịch vụ • Lựa chọn	6, 8, 12, 17
Hà Tĩnh - Khối 12	Tìm Số • Số cặp nhảy • Hành trình cứu trợ	6, 10, 15
Thành phố Huế	Trò chơi với dãy số • Bảng số • Hàng rào • Trồng cây	2, 6, 8, 10, 16
Nghệ An	Mã hàng hóa • Nhiên liệu • Hoa đẹp	3, 13, 16
Quảng Ninh	Trễ hạn • Đếm dãy con • Phần thưởng • Phân tích cây	1, 2, 7, 9
Quảng Trị	Trò chơi điện tử • Thẻ bài • Phần tử thiếu • Bước nhảy không gian	2, 3, 12, 13, 15, 18
Thái Nguyên	Hội Xuân • Phân công công việc • Tối ưu hàm • Hệ thống điện	3, 4, 7, 14, 17
An Giang	Biến đổi • Số đẹp • Dãy con	1, 4, 6
Đắk Lắk	Phương trình • Mật khẩu • Dãy số • Trò chơi • Mạng máy tính	1, 3, 4, 5, 7, 10, 14, 18
Đồng Nai - Bảng A	Phát quà • Hàng rào • Cảm biến • Phi tiêu	1, 6, 8, 10, 16, 18
Đồng Nai - Bảng B	Số may mắn • Hội trại • Thiết bị bay • Số nguyên tố • Robot	5, 6, 8, 12, 13, 18
Đồng Tháp	Số đặc biệt • Bộ ba số nguyên • Kế hoạch xây dựng • Hội thảo	9, 11, 12, 14
Gia Lai	Đếm dãy • Xâu đối xứng • Chọn quà • Tải ứng dụng	10, 12, 14, 15, 16
Hưng Yên	Chia dư k • Lệch nhỏ nhất • Số lớn nhất • Xe cấp cứu	2, 3, 5, 11, 13, 15
Khánh Hoà	Tổng phân số • Dãy tăng liên tiếp • Chia phần • Mảnh đất trồng cây	6, 12, 17, 18
Lào Cai	Ước số • Tìm chuỗi kí tự • Chọn số • Trung vị của dãy số • Gặp gỡ	5, 6, 8, 20, 22, 23
Lâm Đồng	Xâu con • Cạm bẫy • Mật mã • Du lịch	2, 5, 6, 19
Phú Thọ	Dãy số • Đếm số nguyên tố • Xâu con dài nhất • Ăn vặt	3, 6, 8, 12, 21
Quảng Ngãi	Hiệu hai bình phương • Đếm nguyên tố đoạn • Mật mã theo ngưỡng • GCD đoạn	6, 10, 23, 24
Tuyên Quang	Cặp số chẵn • Tổng đoạn • Tương phản • Cứu trợ	6, 8, 11, 13, 24
Vĩnh Long	Xếp thẻ • Chênh lệch • Nhận diện	2, 3, 4, 24

Lê Thanh Nam

GIỚI THIỆU

CÁCH HỌC MỖI KỸ THUẬT TRONG CUỐN SÁCH

Học sinh chưa biết gì không nên bắt đầu bằng việc chép code. Với mỗi kỹ thuật, hãy đi đúng sáu tầng: nhìn dữ liệu cụ thể → mô phỏng bằng tay → viết cách ngây thơ → chỉ ra chỗ lặp thừa → rút ra tính chất → cuối cùng mới viết code tối ưu. Nếu bỏ qua một tầng, học sinh rất dễ thuộc mẫu nhưng không biết khi nào được dùng.

Bước 1. Đọc ví dụ nhỏ và tự tính đáp án bằng tay, chưa nhìn code.

Bước 2. Ghi lại trạng thái sau từng thao tác: chỉ số, giá trị đang giữ, tập/cửa sổ/đỉnh hiện tại.

Bước 3. Viết lời giải trực tiếp nhất bằng tiếng Việt. Chỉ khi học sinh diễn đạt được bằng lời mới chuyển sang C++.

Bước 4. Thay giới hạn lớn nhất vào độ phức tạp để thấy vì sao cách trực tiếp không đủ.

Bước 5. Tìm bất biến hoặc tính đơn điệu làm "giấy phép" cho cách tối ưu.

Bước 6. Viết code hoàn chỉnh, chạy tay lại trên đúng ví dụ vừa dùng, sau đó kiểm tra test biên.



Hình 0.1. Tách bạch "thứ tự phần tử" và "chỉ số C++" ngay từ bài học đầu tiên.

NGUYÊN TẮC SƯ PHẠM

Mọi đoạn code trong phần bổ sung đều có một ví dụ chạy tay và một câu trả lời cho câu hỏi "vì sao đúng?". Công thức quan trọng được đặt trong dấu trong Equation

Lê Thành Nam

C++ TỪ SỐ 0

Dành cho học sinh chưa từng lập trình hoặc còn hổng kiến thức nền

Phần này không giả định học sinh đã biết bất kỳ cú pháp C++ nào. Mục tiêu là đọc được một chương trình đơn giản, tự chạy tay, viết được vòng lặp và mảng trước khi học thuật toán.

■ 0.1. Một chương trình C++ tối thiểu gồm những gì?

Nếu học sinh chưa từng lập trình, hãy coi chương trình là một bản hướng dẫn rất chính xác cho máy tính. Máy chỉ làm đúng những lệnh ta viết, theo đúng thứ tự. Trong lập trình thi đấu, hầu hết chương trình có ba phần: đọc dữ liệu vào, xử lý, rồi in kết quả.

Dòng #include <bits/stdc++.h> nạp hầu hết thư viện chuẩn thường dùng. using namespace std; giúp viết cin, cout, vector, sort thay vì std::cin, std::cout... Hàm main() là điểm bắt đầu chạy chương trình. return 0; báo chương trình kết thúc bình thường.

Dấu chấm phẩy ; kết thúc một câu lệnh. Cặp ngoặc nhọn { } gom nhiều câu lệnh thành một khối. C++ phân biệt chữ hoa và chữ thường: ans và Ans là hai tên khác nhau.

■ 0.2. Biến, kiểu dữ liệu và phép gán

Biến là một ô nhớ có tên. Ví dụ int a = 5; tạo biến a kiểu số nguyên và gán giá trị 5. Khi viết a = a + 2; máy lấy giá trị cũ của a, cộng 2 rồi ghi trở lại a. Dấu = trong lập trình là phép gán, không phải dấu bằng toán học.

Các kiểu cần dùng nhiều nhất: int cho số nguyên cỡ 2×10^9; long long cho số nguyên cỡ 9×10^{18}; double cho số thực; char cho một ký tự; string cho một xâu; bool cho đúng/sai. Khi không chắc tổng hoặc tích có vượt int hay không, ưu tiên long long.

Phép toán cơ bản: +, -, *, /, %. Với số nguyên, a / b lấy phần nguyên của thương và a % b lấy phần dư. Ví dụ 17 / 5 = 3 và 17 % 5 = 2.

■ 0.3. Nhập xuất với cin và cout

cin >> a; đọc một giá trị từ input và cất vào a. Có thể đọc nhiều biến liên tiếp: cin >> n >> k;. cout << ans << "\n"; in kết quả và xuống dòng. Trong đề thi, thứ tự đọc phải khớp chính xác mô tả input.

Với dữ liệu lớn, đặt ios_base::sync_with_stdio(false); và cin.tie(nullptr); ở đầu main để tăng tốc nhập xuất. Không nên dùng endl liên tục vì endl vừa xuống dòng vừa buộc xả bộ đệm.

■ 0.4. Điều kiện if / else

if dùng khi chương trình phải rẽ nhánh. Ví dụ nếu a lớn hơn b thì lấy a, ngược lại lấy b. Điều kiện so sánh thường dùng: ==, !=, <, ≤, >, ≥. Lỗi rất hay gặp của người mới là viết if (a = b) thay vì if (a == b).

Các phép logic: && nghĩa là và, || nghĩa là hoặc, ! nghĩa là phủ định. Luôn dùng ngoặc nếu điều kiện dài để người đọc nhìn được cấu trúc.

■ 0.5. Vòng lặp for và while

Vòng lặp cho phép lặp một công việc nhiều lần. Khuôn for (int i = 0; i < n; i++) chạy i = 0,1,...,n-1. Đây là khuôn duyệt mảng phổ biến nhất. while (điều kiện) lặp chừng nào điều kiện còn đúng.

Người mới phải tập chạy tay vòng lặp: ghi lần lượt giá trị của i và những biến bị thay đổi sau mỗi vòng. Kỹ năng này giúp tìm lỗi nhanh hơn nhiều so với chỉ nhìn code.

Lê Thành Nam

0.6. Hàm - chia bài toán thành các khối nhỏ

Hàm là một đoạn chương trình có tên, nhận dữ liệu đầu vào và có thể trả về kết quả. Ví dụ long long gcd(long long a, long long b) nhận hai số và trả về ước chung lớn nhất. Tách hàm giúp code dễ kiểm tra và dùng lại.

Tham số truyền bằng giá trị tạo bản sao; truyền bằng tham chiếu với dấu & cho phép hàm làm việc trực tiếp trên đối tượng gốc. Với vector lớn, thường truyền const vector<int>& a nếu chỉ đọc để tránh sao chép.

0.7. Mảng, vector và string

vector<int> a(n); tạo một dãy n số nguyên, đánh chỉ số từ 0 đến n-1. a[i] là phần tử thứ i theo chỉ số C++; khi đề đánh số từ 1, học sinh phải quyết định rõ mình sẽ giữ 1-based hay đổi sang 0-based và nhất quán đến cuối.

string s; lưu xâu ký tự. s.size() là độ dài, s[i] là ký tự ở vị trí i. vector có các thao tác push_back, pop_back, size; string cũng có nhiều thao tác tương tự.

0.8. Cách đọc một đề thi lập trình

Đừng viết code ngay. Hãy gạch ra bốn dòng: (1) Input là gì? (2) Output cần gì? (3) Giới hạn lớn nhất là bao nhiêu? (4) Với ví dụ, mình có thể tự tính đáp án bằng tay không? Nếu chưa trả lời được bốn câu này thì chưa nên code.

Sau đó viết cách giải ngây thơ nhất bằng lời. Ước lượng số phép tính. Nếu quá chậm, tìm chỗ lặp lại công việc: tính lại tổng đoạn, quét lại một dải, kiểm tra lại cùng một số, dựng lại cùng một tập... Điểm lãng phí thường chính là manh mối dẫn đến thuật toán tối ưu.

0.9. Chạy tay, debug và kiểm thử

Khi chương trình sai, tạo test cực nhỏ 3 đến 5 phần tử và ghi bảng các biến sau từng bước. Test phải có trường hợp biên: n=1, toàn bằng nhau, toàn âm, có 0, đáp án ở đầu/cuối, giá trị lớn nhất cho phép.

Một kỹ thuật rất mạnh là stress test: viết bản vét cạn chắc chắn đúng cho n nhỏ, sinh nhiều test ngẫu nhiên, chạy cả bản vét cạn và bản tối ưu rồi so sánh. Tuyển tập lời giải THPT cũng dùng tinh thần đối chiếu bản ngây thơ với bản tối ưu để tăng độ tin cậy.

▸ *Chương trình mẫu đầu tiên - đọc n số và tính tổng*

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    long long s = 0;
    for (int i = 0; i < n; i++) {
        s += a[i];
    }

    cout << s << "\n";
    return 0;
}
```

Lê Thanh Nam

BỔ SUNG CHO HỌC SINH

Các mục sau giải thích thêm hàm, phạm vi biến, cách đọc lỗi và khung C++17. Học sinh nên tự gõ lại code, không sao chép bằng chuột.

Mỗi khi gặp một cú pháp mới ở các chương sau, hãy quay lại đối chiếu phần C++ từ số 0 này.

0.8. Hàm (function) - tách một công việc thành khối có tên

Hàm giúp tách chương trình thành phần nhỏ. Kiểu trước tên hàm là kiểu giá trị trả về; void nghĩa là không trả về. Tham số truyền bằng giá trị tạo bản sao; thêm & để truyền tham chiếu và cho phép sửa đối tượng gốc.

Trong thi đấu, nên viết hàm cho các thao tác có ý nghĩa rõ như gcd tự viết, ok(mid), bfs(s), find(x), query(...). Khi lỗi, ta kiểm tra từng khối thay vì một main dài hàng trăm dòng.

0.9. const, phạm vi biến và biến toàn cục

const int MAXN=200005; tạo hằng số không đổi. Biến khai báo trong một cặp { } chỉ tồn tại trong khối đó. Biến toàn cục tồn tại suốt chương trình và mặc định được khởi tạo 0; mảng lớn thường khai báo toàn cục để tránh giới hạn stack.

Không biến mọi thứ thành biến toàn cục. Dùng toàn cục cho cấu trúc rất lớn hoặc dữ liệu nhiều hàm cùng cần; còn biến tạm nên đặt cục bộ để dễ kiểm soát.

0.10. Đọc lỗi biên dịch và lỗi chạy

Compile Error thường do sai cú pháp, thiếu ; sai tên biến/hàm, kiểu không khớp. Runtime Error thường do vượt mảng, chia 0, stack overflow, truy cập top/front của cấu trúc rỗng. Wrong Answer nghĩa là chương trình chạy nhưng logic sai. Time Limit Exceeded thường do độ phức tạp hoặc I/O quá chậm.

Khi gặp lỗi, đọc dòng đầu tiên có ý nghĩa trong thông báo compiler. Không sửa ngẫu nhiên nhiều chỗ cùng lúc; sửa một nguyên nhân rồi biên dịch lại.

0.11. Mẫu khung C++17 nên thuộc nhưng phải hiểu

Khung thi đấu tối thiểu gồm bits/stdc++.h, using namespace std, ios_base::sync_with_stdio(false), cin.tie(nullptr), main. Hai dòng tối ưu I/O phải đặt trước khi dùng cin/cout.

Không cần freopen trừ khi đề bắt buộc đọc/ghi file hoặc muốn tự động hỗ trợ file cục bộ. Trên hệ thống chấm dùng stdin/stdout, cin/cout là đủ.

▸ *Ví dụ hoàn chỉnh - tách hàm kiểm tra số chẵn*

```
#include <bits/stdc++.h>
using namespace std;

bool chan(long long x) {
    return x % 2 == 0;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    long long sum = 0;
    for (int i = 0; i < n; i++) {
        long long x;
        cin >> x;
        if (chan(x)) sum += x;
    }
```

Lê Thành Nam

```
    cout << sum << "\n";
    return 0;
}
```

TỰ KIỂM TRA TRƯỚC KHI SANG PHẦN I

Tự viết chương trình đọc hai số và in số lớn hơn.

Tự viết chương trình tính tổng 1+2+...+n bằng vòng lặp.

Tự đọc n phần tử vào vector và tìm max/min.

Tự giải thích bằng lời từng dòng của chương trình mẫu ở trên.

Lê Thành Nam